

BonusKdTreeDemo

The Sample Project Mini-Cookbook

This FREE booklet is designed to accompany the sample project `BonusKdTreeDemo` from the book [Practical C++ Game Programming with Data Structures and Algorithms](#). It provides additional insights into the code logic, algorithms, and problem-solving approaches used in the implementation of this sample.

In this mini booklet, you'll find how the sample project works and the key ideas behind its design.

Before we begin

You will need the knowledge of Chapter 1 and Chapter 2, or basic understanding of raylib and common data structure like array/list. It's the best time to try out this sample after you finish reading Chapter 2. This booklet will not repeatedly cover anything you need to know already in the book.

Getting project up and running

First, make sure you have pulled the latest updates from GitHub (<https://github.com/PacktPublishing/Practical-C-Game-Programming-with-Data-Structures-and-Algorithms/tree/main>).

Open the Visual Studio solution file included with this book in Visual Studio Community 2022, select the `BonusKdTreeDemo` project as start-up project, rebuild it, and run. You should see the similar scene like [Figure 1](#). You can use arrow keys to move the player character, rotate camera with mouse moving while holding right button, or using mouse wheel to zoom.

Try to use mouse left button to click on any of the green cubes. The cube will turn red and the surrounding cubes within predefined radius will turn blue. The range is rendered as semi-transparent sphere.

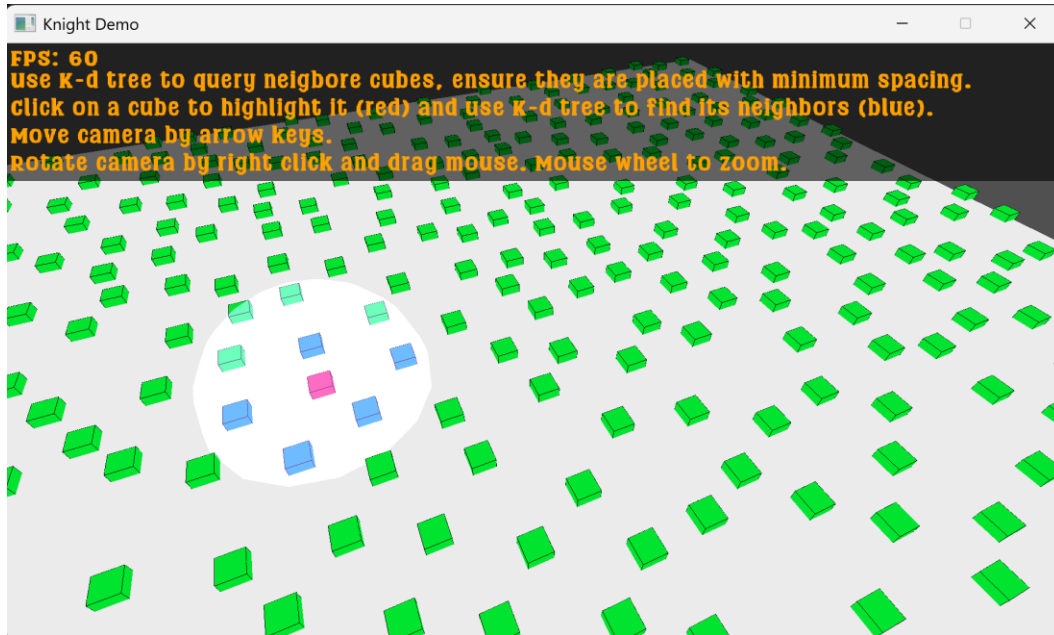


Figure 1 – Running the sample project BonusKdTreeDemo

Tree: a hierarchical data structure

A **tree** is one of the fundamental data structures in computer science. It is widely used for organizing information in a hierarchical manner, making it easy to represent relationships, speed up searches, and structure complex systems such as file systems, compilers, and games.

Thinking of file system of your device, it is one of the most typical examples of a tree data structure. On any storage device, whether a hard drive or a smartphone, there is always a root folder. Under the root, you can store individual files or organize them into folders to classify different types and purposes of data. Within each folder, you can also have child folders.

In Chapter 2, when we studied arrays and lists, our focus was on the ordering of individual elements relative to one another. With a tree data structure, however, the emphasis shifts to the hierarchical relationships between elements.

When describing a tree, we usually refer to each single element as a **node**. Each node typically records the following relationships with other nodes:

- Parent node – The node above it. If the node is the very first node in the tree, it serves as the parent of all other nodes.
- Sibling nodes – Other nodes at the same level as the current node.
- Children nodes – All the child nodes that belong to the current node.

Some implementations only record the first child node, with each node then keeping track of its own siblings. Other implementations may not record sibling nodes at all—in that case, to find a node's siblings, you must check the parent node's list of children.

The folder–file hierarchy of an operating system is a classic example of a tree data structure. But what about in game development?

Starting from **Chapter 1**, we have already used this kind of data structure in our **Demo1** project. Knight's **Scene** is organized using a tree structure, where **SceneObject** is the typical node used to manage and organize hierarchical relationships.

Because many data that can be organized hierarchically can be represented as a tree, the tree has become one of the most widely used data structures:

- Databases & File Systems: organize folders, indexes, and queries.
- Compilers: parse code into abstract syntax trees.
- Search & Sorting: BSTs allow fast searching ($O(\log n)$ on average).
- AI in Games: decision trees and behavior trees model non-player character (NPC) behavior.
- Graphics & Game Engines: scene graphs, spatial partitioning trees (BSP, quadtrees, octrees) optimize rendering and collision detection.

Here, we will focus only on applications related to games. For examples of how trees are used in other areas of computer science, you can refer to most data structure textbooks or online tutorials.

Trees give us a way to represent non-linear, hierarchical relationships. Whether it's an in-game AI decision tree, a filesystem browser, or a rendering optimization, understanding trees is a cornerstone for both theory and practical applications in game programming too.

Building a binary tree

A **binary tree** is a hierarchical data structure where each node has at most two children, commonly referred to as the left child and right child. It's a special case of a tree data structure and forms the foundation for many efficient algorithms and specialized trees such as **k-d trees**, **heap** and **binary search tree (BST)**.

Figure 2 illustrates a typical binary tree. Each node has at most 2 child nodes. We can simply name them left child node and right child node. Like all tree data structures, it start with a root node with a single value 35, but can be any kind of data type. The **level** or **depth** refers to how many steps from the root node to traverse to the node itself.

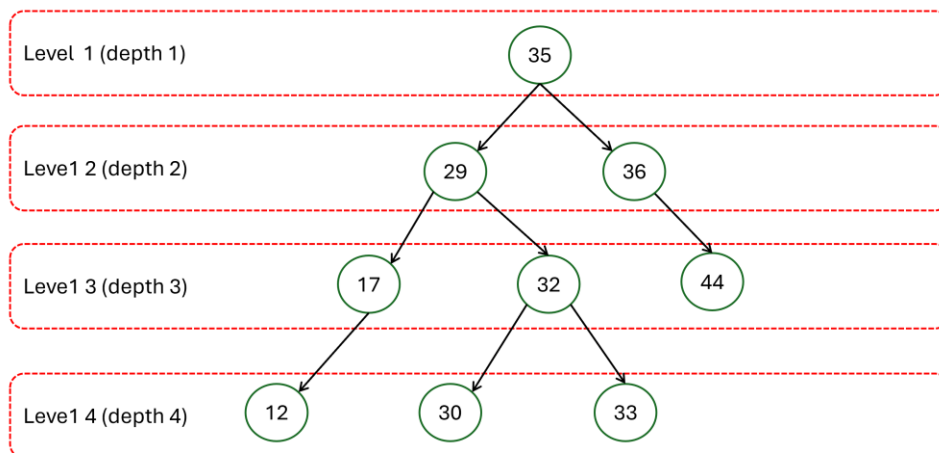


Figure 2 - A typical binary tree only has at most 2 children per node.

In Figure 2, the nodes in level 4 without any children are usually called leaf nodes. Ideally, we like the tree to be “balanced” - to have majorities of the leaf nodes roughly at same depth. If one branch has children’s nodes twice or triple depth compared to rest of the tree branch nodes, it’s unbalanced. You can think of the most extreme case of unbalanced trees which has only one branch like **Figure 3**:

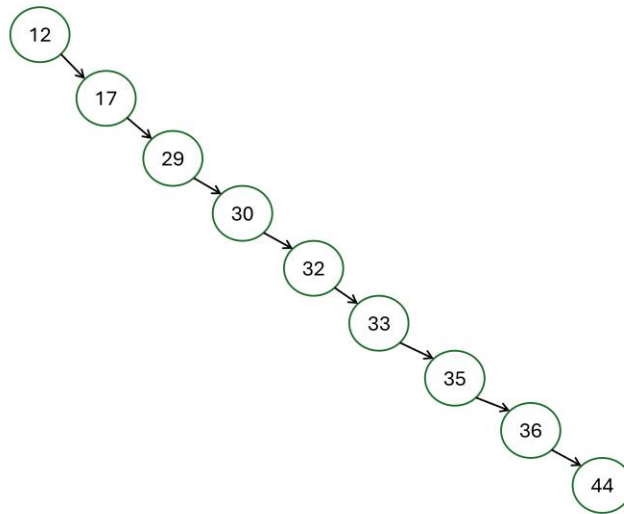


Figure 3 – The most unbalanced tree can hardly call itself a tree

Do you notice the similarity between the trees in **Figure 2** and **Figure 3**? In both cases, the left child node always has a smaller value than its parent, while the right child node always has a larger value. The difference lies in how the same set of data is organized: **Figure 2** shows an **better balanced** binary tree, while **Figure 3** shows the **worst-case unbalanced** tree.

Now, suppose you want to check whether the tree contains a node with the value 44. This is a common operation—querying whether a particular element exists in a collection—like searching an array, which we covered in **Chapter 2**. Because of how a binary search tree is structured, we know that all nodes in the left subtree hold smaller values, while all nodes in the right subtree hold larger values.

In the better balanced tree of **Figure 2**, you would only need to traverse at most **4 levels** to determine whether the value 44 is in the tree. In the worst-case unbalanced

tree of **Figure 3**, however, you would need to go all the way down to the last child node before reaching a conclusion.

In fact, building a binary tree like **Figure 3** offers no real advantage over using a simple array and performing a straightforward linear search to check whether the value 44 exists.

There are many different applications of binary tree during game development, and one particular use is to organize your game world in an hierarchical way to speed up rendering and collision detection. Like the famous **Quake engine** used binary tree to partition the game level into recursively divided convex sets (**binary space partition, BSP**), helping in rendering complex 3D environments efficiently.

Using K-dimensional tree

In this sample cookbook, we will focus on a special type of binary tree only: **K-d tree**.

K-d tree is a variant of binary trees used for efficient and quick nearest-neighbor searches, collision detection, and ray casting.

Imagine an empty, open-world scene where you need to spawn 20 defense towers on the ground for the player to fend off threats. This requirement comes with a few specific needs:

- The towers must be placed with at least a configurable minimum distance between them, so they don't bunch up or overlap, and so coverage doesn't become too sparse in some areas (which would create defensive gaps). No matter where the player spawns, there should be a tower at a reasonably similar distance nearby.
- During gameplay, each tower should be able to quickly find other towers or the player character within a given radius—crucial for implementing tower defense AI.

If there are only 20 towers, you don't need any special data structure: a simple array of 20 elements is enough to store tower states, and a single for loop is fast enough to be able to scan the whole array to find neighbors within a distance.

However, if—like in the `BonusKdTreeDemo` project—you need to search around hundreds of objects (e.g., finding nearby cubes among 400 or 2000 of them), pairwise checks become impractical. This is where a k-d tree, which excels at spatial nearest-neighbor and range queries, becomes useful.

Before diving into the code, let's briefly define a k-d tree:

A k-d tree (short for *k-dimensional tree*) is a special kind of binary search tree designed for organizing points in a k-dimensional space.

- Each node represents a point in k-dimensional space.
- At every level of the tree, the space is split by a divider (an axis line in 2D, or a plane in 3D, etc.) perpendicular to one of the coordinate axes.
- The splitting axis cycles with depth (e.g., $x \rightarrow y \rightarrow z \rightarrow x$ again in 3D).

Let's use a simple example to see how the tree is built. Consider a map with 6 ghosts characters on x-z plane (y axis point upward) like *Figure 4*:

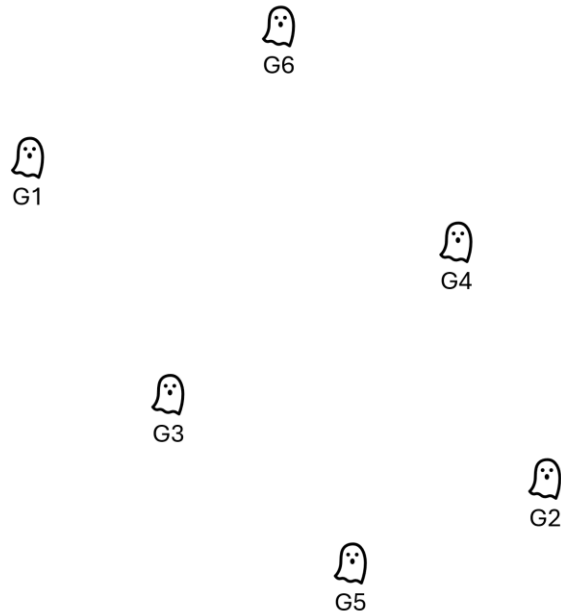


Figure 4 - a x-z plane with 6 ghost characters, G1~G6

Let's begin by splitting the map along the horizontal X-axis. After sorting the positions by their X values (from smallest to largest), the order is [G1, G3, G6, G5, G4, G2]. To keep the tree as balanced as possible, we select the X value of the middle element, G6, as the first splitting line. We then create the root node X1, which divides all the ghosts into two segments:

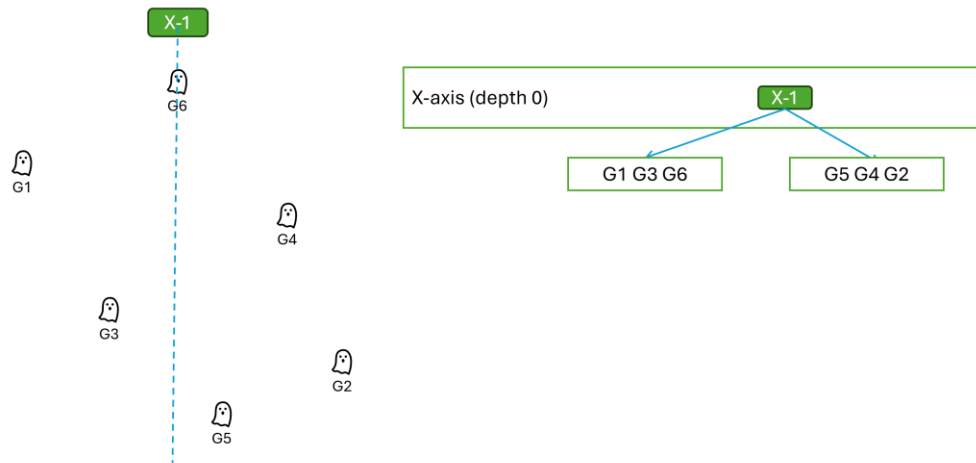


Figure 5 – Using first root node to split x-axis

Next, we switch to split the Z-axis for both segments previously split by node X1:

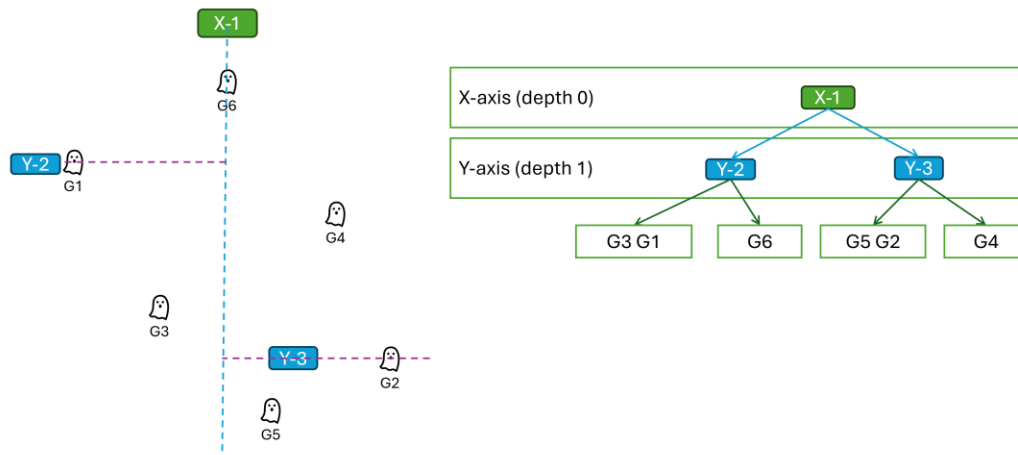


Figure 6 – Continuing to build the tree nodes by splitting z-axis

Figure 6 illustrates how to continue splitting the nodes from the previous step. For the [G1, G3, G6] segment, sorting along the Z-axis gives the new order [G3, G1, G6]. We then choose the Z-position of the middle element, G1, as the split line to create child node Y2.

Similarly, for the other segment [G5, G4, G2], sorting along the Z-axis results in [G5, G2, G4]. We choose the Z-position of the middle element, G2, as the split line to create child node Y3.

Let's continue to split nodes until there is no further nodes to split:

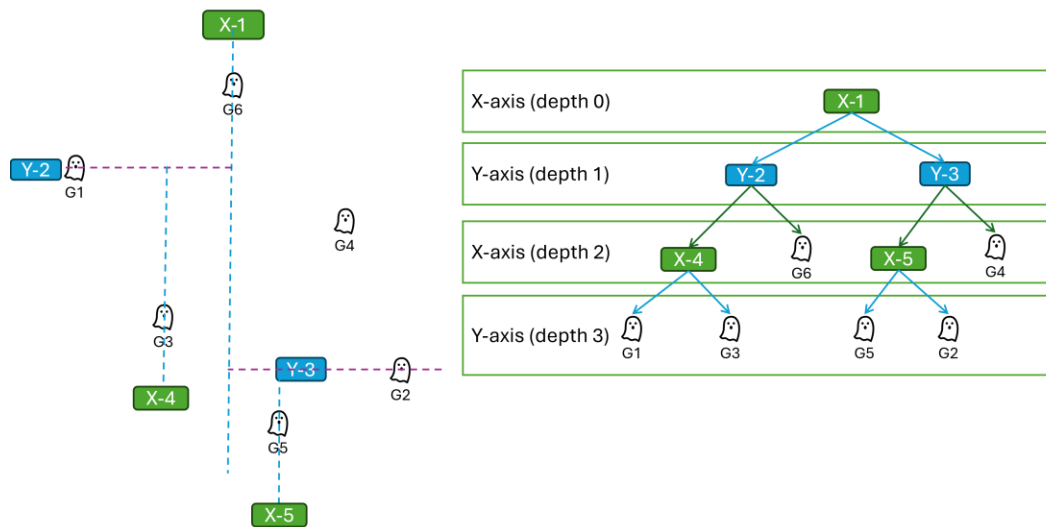


Figure 7 – The final k-d tree built by repeating previous process

There are two advantages of such way to partition a space:

- Efficient search: Finds nearest neighbors, range queries, or points within a region much faster than brute force.
- Balanced partitioning: Divides space into halves recursively, creating a binary structure.

So, in short:

A **k-d tree** is a binary tree that recursively splits multi-dimensional space, making spatial queries (like “who’s nearest?”) very fast.

This brings the following benefits in various topics of game development:

- Spatial partitioning: Manage thousands of objects in 2D/3D space.
- Collision detection: Quickly find nearby objects to test for collisions.
- Nearest-neighbor queries: AI agents searching for the closest resource, target, or enemy.
- Rendering optimizations: Speeding up ray casting and visibility checks.

Let’s check out a real example project in the next section.

Putting the K-d tree in action

Let’s get back to the sample project `BonusKdTreeDemo`. The project takes the advantage of K-d tree in two ways:

- All the `Cubes` are placed at a configurable minimum distance `MIN_SPACING`, so they will not appear overlapped or too close to each other, this is good when randomly place large numbers of entities on a large terrain.
- Quickly locate surrounding Cubes with a given distance range. This is useful for combat AI to identify potential targets and becomes even more valuable when dozens or even hundreds of AI agents need to perform target searches simultaneously.

Now it’s time to deep down to the real code. This sample follows the same pattern of other Knight samples. `BonusKdTreeDemo.cpp` is the main entry and main Knight application class. The K-d tree is implemented in `KdTree2XZ.cpp` (and header file in `KdTree2XZ.h`).

The provided code implements a 2D k-d tree structure, specifically optimized for X and Z coordinates (a common setup in 3D environments where Y is the vertical axis). This data

structure is used for partitioning a set of points in a 2D space to efficiently perform tasks like range searching and nearest neighbor searching.

Defining the node

First, let's define our node in `KdTree2XZ.h`:

```
struct KdNode2
{
    int index;

    int axis; // 0=x, 1=z

    KdNode2* left;

    KdNode2* right;
};
```

In the `KdNode` structure, the `axis` field specifies whether the current split is based on the X-axis or the Z-axis. Each node also contains pointers to its two child nodes: `left` and `right`.

Building the K-d tree

The `Start()` in `BonusKdTreeDemo.cpp` calls `GenerateCubesNoOverlap()` to create and place the cubes. The core function to build the k-d tree is the `Build()`:

```
void KdTree2XZ::Build(const std::vector<Vector3>& points)
{
    Clear();

    pts = &points;

    indices.resize((int)pts->size());

    for (int i = 0; i < (int)pts->size(); ++i) indices[i] = i;

    nodes.reserve(pts->size());

    root = BuildRange(0, (int)indices.size(), 0);
}
```

And the `BuildRange()` function, which is called internally:

```
KdNode2* KdTree2XZ::BuildRange(int l, int r, int depth)
{
    if (l >= r)
        return nullptr;

    int axis = depth & 1;

    int m = (l + r) / 2;

    std::nth_element(indices.begin() + l, indices.begin() + m,
indices.begin() + r, [&](int a, int b) {
        return coordXZ(a, axis) < coordXZ(b, axis);
    });

    KdNode2* node = MakeNode();
    node->index = indices[m];
    node->axis = axis;
    node->left = BuildRange(l, m, depth + 1);
    node->right = BuildRange(m + 1, r, depth + 1);

    return node;
}
```

The `axis` is determined by `depth & 1`. This simple trick ensures that the splitting `axis` alternates value 0 (X-axis) for depths 0, 2, 4... and value 1 (Z-axis) for depths 1, 3, 5....

The most important step here is how to choose a good value to split the axis, we need to find the median number along the chosen axis. The C++ standard library function `std::nth_element` partitions the sub-array of indices (from `l` to `r`) so that the element at index `m` is the same as it would be if the entire array were fully sorted along the current axis. This element becomes the median point for the split.

The function then recursively calls `BuildRange()` on the left half (`l` to `m`) and the right half (`m+1` to `r`), incrementing the depth at each level.

Keeping minimum distance with K-d tree

Each time a new cube is generated, a random candidate position is chosen first:

```
Vector3 candidate{ d(rng), 0.0f, d(rng) };
```

Next, we use the `RadiusSearchXZ()` function of the `KdTree2XZ` class to quickly find all cubes that have already been placed within a certain distance of this candidate position:

```
std::vector<int> neigh;

tree.RadiusSearchXZ(candidate, (MIN_SPACING + CUBE_SIZE) * (MIN_SPACING + CUBE_SIZE), neigh);
```

We then check whether this position is too close to any of the nearby cubes:

```
bool ok = true;
for (int idx : neigh) {
    if (Dist2XZ(candidate, positions[idx]) < minR2) {
        ok = false;
        break;
    }
}
if (!ok)
    continue; // if this is not an idea position, try again!
```

How many times should this process be repeated at most? In the current program, we set the limit to 2,000 attempts. If all 2,000 fails, no new cube will be generated.

Enumerating neighbor cubes nearby

In the `BonusKdTreeDemo`, when the user selects a cube with the mouse, we also use the `RadiusSearchXZ()` function to quickly search for nearby cubes and change their color to blue:

```
cubes[selectedIdx].color = RED; // highlight clicked cube
// Find neighbors within NEIGHBOR_R and turn them BLUE
```

```

std::vector<int> neighbors;

tree.RadiusSearchXZ(cubes[selectedIdx].pos, NEIGHBOR_R * NEIGHBOR_R,
neighbors);

for (int idx : neighbors) {
    if (idx == selectedIdx)
        continue;

    cubes[idx].color = BLUE;
}

```

Searching nodes inside the K-d tree: RadiusSearchXZ()

Inside the `RadiusSearchXZ()` function, the recursive function `Search()` is called.

The recursive `Search()` function traverses the tree to find all points within a squared distance of `radius2` from the query point `q`:

```

void KdTree2XZ::Search(KdNode2* node, const Vector3& q, float radius2,
std::vector<int>& out) const
{
    if (!node)
        return;

    // Current Point Check
    const Vector3& p = (*pts)[node->index];
    if (Dist2XZ(q, p) <= radius2)
        out.push_back(node->index);

    // Determine Traversal Order
    float qv = (node->axis == 0) ? q.x : q.z;
    float pv = (node->axis == 0) ? p.x : p.z;
    KdNode2* first = (qv < pv) ? node->left : node->right;
    KdNode2* second = (qv < pv) ? node->right : node->left;

    // Traverse Closer Subtree
    Search(first, q, radius2, out);
}

```

```
float planeDist2 = (qv - pv) * (qv - pv);  
  
// Pruning Check  
  
if (planeDist2 <= radius2)  
    Search(second, q, radius2, out);  
  
}
```

In the above code snippet:

1. **Current Point Check:** If the distance between the node's point p and the query point q is less than or equal to `radius2`, the node's index is added to the out vector.
2. **Determine Traversal Order:** It determines which subtree (first) is closer to the query point q based on the current splitting axis, and which is further (second).
3. **Traverse Closer Subtree:** It always recursively calls `Search()` on the first (closer) subtree.
4. **Pruning Check:** This is the key to efficiency. `planeDist2` is the squared distance from the query point q to the splitting hyperplane. If this squared distance is greater than `radius2`, it means the search sphere does not cross the plane, and the second (further) subtree cannot contain any points within the search radius. In this case, the second subtree is pruned (not searched). If `planeDist2` is less than or equal to `radius2`, the sphere might intersect the further half-space, and the second subtree must be searched.

Important constants

In the sample, several important constants are defined:

```
// change as you like, number of total Cubes  
static constexpr int TOTAL_CUBES = 400;  
  
// size of the cube  
static constexpr float CUBE_SIZE = 1.2f;  
  
// hard no-overlap
```

```
static constexpr float MIN_SPACING = 2.5f * CUBE_SIZE;
// half-extent of placement area
static constexpr float AREA_HALF = 40.0f;
// radius for "neighbors"
static constexpr float NEIGHBOR_R = 6.0f;
```

You can try modifying these constants and observe how the changes affect the program's behavior.

Conclusion

We used the `BonusKdTreeDemo` project to demonstrate some use cases of K-d tree in game development. This is a special case of binary trees useful to query neighbor objects fast.

Compared to linear search of whole list of `Cube` objects, the average search operation is around $O(\log n)$ where n is the total number of `Cube`. The worst case can be like $O(n)$ if the tree is extremely unbalanced.

You can simply grab K-d tree class from `KdTree2XZ.cpp/KdTree2XZ.h` to your own project without further dependency other than raylib (for basic types like `Vector3`, etc.)

Challenging yourself

In the `BonusKdTreeDemo`, all `Cube` objects are pre-generated, and their positions remain fixed throughout the program's execution. As a result, the k-d tree only needs to be built once in the `Start()` function.

However, if some or all cubes are allowed to move, then the k-d tree must be partially or fully rebuilt during each `Update()` cycle.

Since rebuilding takes time, in practice there's no need to rebuild every frame. Instead, you can adopt a strategy such as rebuilding every n frames or use a more advanced adaptive rebuild strategy—only triggering a rebuild when the number of moving cubes exceeds a specified threshold.

To further optimize performance, you can separate all 2,000 entities into two groups: **static entities** (which remain fixed in position throughout the game session) and **dynamic entities** (which can move).

For example, imagine having 1,000 defense towers placed permanently on the terrain and another 1,000 monsters wandering around. Instead of building a single k-d tree with all 2,000 nodes and rebuilding it every frame, you can construct two separate k-d trees:

- One tree for the 1,000 static defense towers, which never needs rebuilding.
- Another tree for the 1,000 wandering monsters, which is rebuilt only when necessary.

This approach significantly reduces the amount of data that must be rebuilt, improving efficiency. It gets more benefit when you have a lot static objects but only a few moving ones.

But is this really a good idea? Or under what circumstances this could be a good idea? I'll left this as a thinking / experiment exercise.

Happy coding!